Universidad Nacional Autónoma de México

Facultad de Ingeniería
Sistemas Operativos

Therac-25:
una condición de carrera y un desbordamiento de 8 bits

Cómo dos errores de concurrencia y de aritmética provocaron sobredosis de radiación, y cómo se evitan hoy.

Jessica Estefani Perez Baez
Noviembre de 2026

01 · Introducción

El caso Therac-25

El Therac-25 fue un acelerador lineal utilizado en radioterapia, fabricado por **Atomic Energy of Canada Limited (AECL).** Entre junio de 1985 y enero de 1987, estuvo implicado en varios accidentes graves por sobredosis de radiación.

Periodo de los accidentes:
junio de 1985 – enero de 1987

¿Qué analizaremos?

- **Condición de carrera:** acceso concurrente a datos compartidos sin sincronización.
- **Desbordamiento de 8 bits:** un contador que regresaba a cero (overflow).
- **Lección:** la importancia de validar configuraciones y contar con mecanismos de seguridad independientes.

Objetivo de la exposición

Explicar cómo ocurrieron estos dos errores a nivel de programación, su relación con los sistemas operativos y cómo se evitan en la **actualidad.**

02 · ¿Cómo funcionaba el Therac-25?

Modos de tratamiento

Electrones

- Haz directo de 5–25 MeV.
- Imanes de barrido que distribuyen el haz sobre el área a tratar.

Rayos X

- Electrones de 25 MeV que impactan un blanco metálico.
- Requiere una corriente mucho mayor.

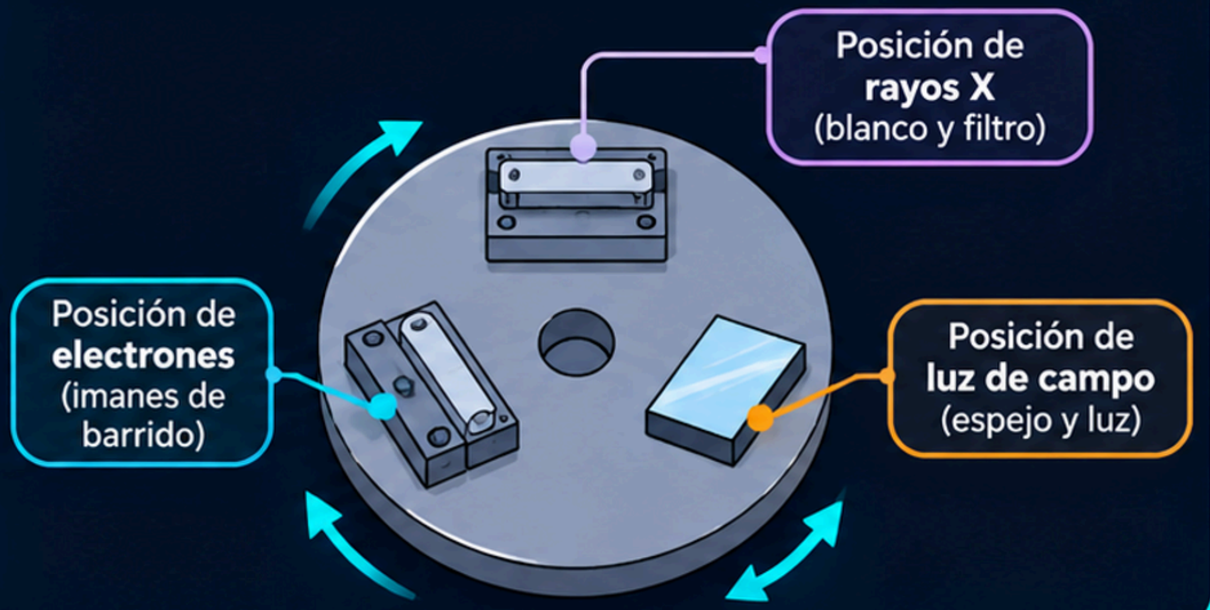
Tercera posición: luz de campo

Solo tiene un espejo y una luz para acomodar al paciente.

No debía utilizarse para irradiar.

Mesa giratoria (*Turntable*)

Colocaba distintos accesorios en la trayectoria del haz según el **modo** de tratamiento seleccionado.



Tres microinterruptores detectaban en cuál de las tres posiciones estaba la mesa.

03 • Sistemas Operativos: arquitectura del software

Plataforma del sistema

- Computadora: PDP-11/23
- Lenguaje: Ensamblador
- Sistema: Tiempo real (*real-time*)

Características

- Controlaba el tratamiento, el haz, la mesa y la seguridad.
- Ejecución concurrente de varias tareas.
- Planificación apropiativa cada 0.1 s (100 ms).

Tareas principales del software

El sistema se organizaba en varias tareas que se ejecutaban de forma concurrente y se comunicaban mediante variables compartidas en memoria.

Planificador
Apropiativo (cada 0.1 s)

Treat
Supervisa el tratamiento en 8 fases (Tphase).

Servo
Controla la emisión del haz, la dosis y los movimientos.

Housekeeper
Gestiona interbloqueos, límites y mensajes.

Teclado
Procesa la entrada del operador y actualiza la configuración.

Problema central: las tareas se comunicaban mediante variables compartidas en memoria.

- Las operaciones de revisar y modificar (test/set) no eran indivisibles.
- Sin sincronización, el resultado dependía del orden de ejecución y de las interrupciones del planificador.

04 • Condición de carrera

¿Qué es?

Es un problema de concurrencia que ocurre cuando **dos o más tareas acceden y modifican un mismo recurso** sin **sincronización,** y el resultado depende del orden en que se ejecutan.

¿Cómo ocurrió en el Therac-25?

En el Therac-25, la tarea de control de movimiento (mesa giratoria) y la tarea de control del haz (radiación) compartían variables. Si el operador cambiaba el modo de tratamiento rápidamente (por ejemplo, de **rayos X** a **electrones)** y corregía la posición de la mesa en ese momento, ambas tareas podían ejecutarse al mismo tiempo y quedar en un **estado inconsistente.**

Interacción entre tareas

1. El operador cambia el modo de tratamiento.
2. Se ejecutan al mismo tiempo las tareas de movimiento y del haz.
3. La mesa y el haz quedan **desincronizados.**

Consecuencia

El equipo podía irradiar al paciente con un haz de alta intensidad, en una posición incorrecta y sin las protecciones adecuadas.

05 • Desbordamiento de 8 bits *(overflow)*

¿Qué es?

Ocurre cuando el resultado de una operación aritmética **excede el valor máximo** que puede representarse con un número de 8 bits (0–255), y el contador **vuelve a cero.**

¿Cómo ocurrió en el Therac-25?

El software utilizaba un contador de 8 bits para calcular y verificar la dosis de radiación. En ciertos casos, el valor de la dosis rebasaba 255, provocando un **desbordamiento** que **reiniciaba el contador a 0.**

Ejemplo simplificado

Valor máximo en 8 bits: **255** + Se añade: **1** = Resultado (real): **256** → En 8 bits (overflow): **0**

Consecuencia

El sistema interpretaba una **dosis de 0 unidades,** por lo que no se detenía la irradiación y continuaba emitiendo el haz.

En resumen: el desbordamiento de 8 bits hizo que el sistema creyera que la dosis era válida y segura, cuando en realidad era mucho mayor a la permitida.

06 • Los accidentes del Therac-25

Incidentes registrados

Entre 1985 y 1987 se documentaron al menos seis casos de sobredosis de radiación, de los cuales tres están ampliamente estudiados:

Marzo de 1985
Hospital en Yakima, Washington.

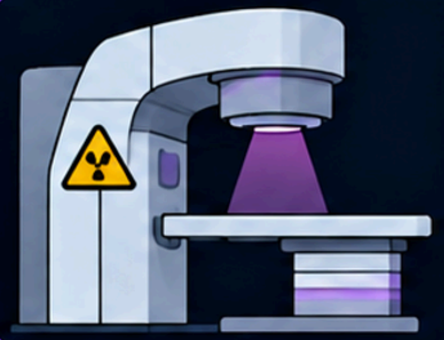
Julio de 1985
Hospital en Yakima, Washington.

Enero de 1987
Hospital en Tyler, Texas.

Secuencia del fallo (caso de Tyler, 1987)

Ocurrió cuando se cambiaba rápidamente del modo de rayos X al modo de electrones, lo que activó la **condición de carrera.**

1. El operador cambia el modo de tratamiento rápidamente.
2. Dos tareas se ejecutan al mismo tiempo y comparten datos.
3. La dosis se calcula con un contador de 8 bits, que se **desborda.**
4. El sistema no detecta el error y se irradia al paciente con una dosis muy alta.



Dosis recibida

El paciente de Tyler recibió aproximadamente **15,000** a **20,000 rad**, cuando la dosis prescrita era de **200 rad.**

Resultados

- Lesiones graves por radiación.
- Quemaduras internas y externas.
- Fallecimiento de varios pacientes días o semanas después.

Causa raíz

La combinación de:

- Una **condición de carrera.**
- Un **desbordamiento de 8 bits.**

Permitió que el equipo aplicara una dosis miles de veces mayor a la indicada.

07 • Accidente de Yakima (17 de enero de 1987)

¿Qué ocurrió?

El 17 de enero de 1987, en el Hospital de Yakima, Washington, una paciente recibió una sobredosis masiva de radiación debido a un fallo en el software del Therac-25.

La causa técnica

8 bits

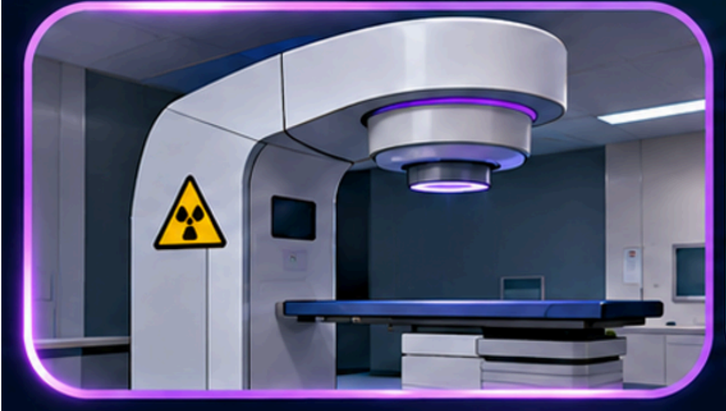
Un contador de 8 bits (Class3) se incrementaba en cada tratamiento.

256

En la pasada 256, el contador se desbordó y regresó a 0, por lo que **no se ejecutó la comprobación de seguridad** del colimador.

Secuencia del fallo

1. El equipo llevaba 255 tratamientos (Class3 = 255).
2. Se inició un nuevo tratamiento (el contador pasó a 0).
3. El operador presionó **«set»** para cambiar al modo de electrones.
4. La comprobación del colimador **no se ejecutó** y la mesa quedó en posición de luz de campo.



Resultado

La paciente recibió una dosis estimada de **16,000 a 20,000 rad** (entre 160 y 200 Gy), cuando la dosis prescrita era de **200 rad** (2 Gy) aproximadamente.

Consecuencias

- Quemaduras graves internas y externas.
- Complicaciones en los días siguientes.
- Fallecimiento de la paciente semanas después.

En resumen: el desbordamiento del contador de 8 bits evitó una verificación crítica, permitiendo que el equipo aplicara una dosis miles de veces mayor a la indicada.

08 • Comparación de los dos fallos

Accidente de Tyler (1986)

Fecha: 21 de marzo de 1986.

Fallo principal:
Condición de carrera (concurrencia).

¿Qué ocurrió?
El cambio rápido de modo y la actualización de la mesa dejaron al sistema en un estado inconsistente, permitiendo emitir el haz sin las verificaciones correctas.

VS

Accidente de Yakima (1987)

Fecha: 17 de enero de 1987.

11111111 ↓ 00000000

Fallo principal:
Desbordamiento de 8 bits (contador Class3).

¿Qué ocurrió?
En la pasada 256, el contador regresó a 0 y se omitió la comprobación del colimador, permitiendo la emisión del haz completo de electrones.

Punto en común

Ambos accidentes se debieron a errores en el software del Therac-25, sin fallas mecánicas.

Aspecto	Tyler (Condición de carrera)	Yakima (Desbordamiento de 8 bits)
Tipo de error	Concurrencia (acceso no sincronizado).	Aritmético (contador de 8 bits).
Efecto	Configuración incoherente del haz y la mesa.	Omisión de la comprobación del colimador.
Resultado	Sobredosis de radiación al paciente.	Sobredosis de radiación (hasta ~20,000 rad).

En resumen: ambos accidentes tuvieron causas distintas, pero comparten la misma raíz: errores en el software y la falta de mecanismos de sincronización y verificación en un sistema crítico.

09 • Impacto y lecciones aprendidas

Impacto en los pacientes

- Dos pacientes recibieron sobredosis masivas de radiación (**≈16,000 a 20,000 rad, ~200 Gy**).
- Ambos sufrieron quemaduras graves, complicaciones internas y fallecieron días después.

Therac-25

Lecciones para los S.O.

- La **concurrencia** debe manejarse con mecanismos de sincronización (mutex, semáforos, monitores).
- Se requiere **validación de datos** y verificación de estados antes de ejecutar acciones críticas.
- Un sistema operativo en **tiempo real** debe garantizar la consistencia y el orden correcto de las tareas.

Recomendaciones de prevención

1. Sincronizar el acceso a datos compartidos.
2. Evitar desbordamientos con variables de mayor tamaño y validación.
3. Implementar verificaciones de seguridad independientes del software.

Principales aprendizajes

- El software puede ser la causa de fallas graves en sistemas de alta criticidad.
- Se necesita una combinación de buenas prácticas de ingeniería, pruebas rigurosas y análisis de riesgos.
- La seguridad en sistemas médicos no debe depender solo del software.

En conclusión: los accidentes del Therac-25 mostraron que un error en el software, ya sea por concurrencia o por un desbordamiento, puede tener consecuencias fatales. Estas fallas impulsaron una mayor atención a la confiabilidad, verificación y seguridad en el desarrollo de sistemas críticos.

10 · Conclusiones

Conclusión general

El caso del Therac-25 demuestra que los errores de software, especialmente en sistemas concurrentes y de tiempo real, pueden tener **consecuencias fatales** cuando no existen mecanismos adecuados de verificación y seguridad.

Therac-25

Principales aprendizajes

- La **concurrencia** y la sincronización deben diseñarse y probarse con rigor.
- Es indispensable **validar datos** y prevenir desbordamientos.
- Se requieren **capas de seguridad** independientes del software.
- La **verificación** y las pruebas exhaustivas son esenciales en sistemas de alta criticidad.

Relevancia en los Sistemas Operativos

- Evidencia la importancia de la **gestión de procesos** y la sincronización.
- Muestra los riesgos de la **concurrencia** sin control.
- Refuerza la necesidad de **sistemas confiables** en aplicaciones críticas.

Reflexión final

Más allá del avance tecnológico que representaba, el Therac-25 nos recuerda que la **confiabilidad del software** es tan importante como el diseño del hardware, y que en sistemas donde la vida de las personas está en juego, la ingenieriá debe priorizar la seguridad, la validación y la prevención de fallas.

Referencias bibliográficas

Libros

- Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts*. 10.ª ed. Wiley.
- Wolf, G. et al. (2015). *Fundamentos de sistemas operativos*. UNAM.

Artículos

- Leveson, N. G. y Turner, C. S. (1993). An Investigation of the Therac-25 Accidents. *IEEE Computer*.
- Leveson, N. G. (2017). The Therac-25: 30 Years Later. *IEEE Computer*.
- Serebryany, K. e Iskhodzhanov, T. (2009). ThreadSanitizer. *Google Research*.

Normas y estándares

- IEC 62304 (2015). *Medical device software — Software life cycle processes.*
- IEC 60601-2-1 (2020). *Medical electrical equipment — Particular requirements for electron accelerators.*
- ISO 14971 (2019). *Medical devices — Application of risk management to medical devices.*